DTD Battleground: Decisions and Trade-offs

Written Oct 2026 before the event. Updated Nov 2026 with event results.
This is the story of how we found problems, what we chose, and why. Numbers marked `[not measured]` are not measured yet, so I don't claim them.

> **Test setup for every number below:** single laptop, `k6` and the server on the same machine, through localhost. This tells us about the server's code, not about venue Wi-Fi.
> **Event:** Oct 26, 2026. 13 teams, 39 participants, TP-Link router network. Started about 5:30 pm (planned 5:00), ended 6:40 pm. No website problems.

1. Understand what the system looks like

One Node server, one copy of the game state in memory, and one `state.json` file on disk. The server also serves the built React site, so one process runs the whole event.

```
phones/laptops -> one Node server -> state in memory -> state.json
```

This is simple, and for one server on one network it works well.

2. Load test and find the real problem

My friends ran `k6` first, then I ran the same scripts on my own laptop so I wasn't trusting screenshots.

Test	Result
Polling only (GET `/api/state`), up to 1000 simulated users	p95 about 2-4 ms across runs; latest 1.92 ms (screenshot)
Registration burst, 150 users, run 1	register p95 11.8 s, 17.8% failed, poll p95 12.6 s
Registration burst, 150 users, run 2	register p95 4.4 s, 0% failed, poll p95 4.2 s

What this told me:

- Polling alone is cheap (about 2-4 ms). During registrations, the same polls took seconds. So polls were waiting behind something else.
- The two burst runs differed a lot, so one run is not a baseline. I did not control test conditions well (whether `state.json` was reset, what else was open). I treat these numbers as direction, not final proof.

- The 17.8% failures happened in run 1 and did not show up again in run 2. I don't know the cause. **[not measured]**

Hypothesis (not proven before the fix): Node runs our JavaScript on one thread, and two registration steps were blocking it:

1. Synchronous password hashing (`scryptSync`). Hashing is slow on purpose, and the sync version freezes the whole server while it runs.
2. Rewriting the whole `state.json` after every request.

3. Decision: fix the code, don't switch to Bun

Bun is another JavaScript runtime that can be faster at HTTP. I thought about switching and decided no:

- Bun is a different runtime, but I had not tested whether it would avoid the blocking, and I chose not to risk a migration a week before the event.
- It adds compatibility risk, and we have little experience with it, a week before the event.

Rule I used: fix what the code does before changing the tool. Bun can be an experiment after the event, with before and after numbers.

4. Decision: async hashing

I decided to switch hashing to async `crypto.scrypt`, which runs on Node's thread pool instead of the main thread; Chiranjeevi implemented it with Claude. Login uses it too (a blocking problem I had initially missed).

The risk: once there's an `await`, other requests can run in the middle of a handler. The duplicate-name check could pass for two requests at the same moment. So:

1. hash first (slow, awaited),
2. then re-check the name and insert with no await in between.

Node only switches between requests at an `await`, so the check and the insert happen as one step.

5. Decision: batched, one-at-a-time disk saves

Problems with saving after every request:

- It is slow when requests arrive in bursts.
- Overlapping async writes can finish out of order, so an older snapshot can overwrite a newer one. A friend raised this, and it is a real risk.

Original design (replaced by group commit):

- State in memory is the source of truth. The file is a copy.
- A "dirty" flag marks that something changed.
- Saves are batched with a 250 ms wait, and only one save runs at a time.
- Each save takes a fresh snapshot of memory when it starts, so a later save is always at least as new as an earlier one.
- It writes to a temp file and renames it over `state.json`. A rename is atomic, so a crash leaves the old complete file or the new complete file, never half of one.
- It flushes on shutdown (Ctrl+C).

Trade-off I accepted: responses are fast, but a sudden crash (power cut, hard kill) can lose the last 250 ms or so of changes. For a quiz where an organiser can ask a team to redo something, that is fine.

Later change (Update): I decided to replace the 250 ms batching with group commit; Chiranjeevi implemented it with Claude. Every action now waits for the write containing its change before replying, so a team is never told 'Correct' or 'Bought' for something a crash could lose. The batching window is 50 ms. Cost: each action is slower by up to about 50 ms plus one write.

Group commit protects against a process crash or hard kill. A power cut can still lose the latest write.

6. Decision: failure handling and backpressure

My friends asked two good questions: what if the disk fails, and what if hashing eats the CPU?

- **Disk failures:** If a save fails, the old valid `state.json` stays untouched. The server records the error, shows a warning in the admin dashboard, and retries with increasing delays (1 s, 2 s, 4 s, up to 30 s).
- **Hash overload:** Only 4 hashes run at once (matching Node's default thread pool). At most 32 more wait at first. After 75 phones signing in at once produced 39 'busy' replies, I raised the limit to 256 (`HASH_QUEUE_LIMIT`; not re-tested). Trade-off: the last sign-ins wait longer instead of being rejected.

Known gaps (verified in code):

- Corrupt state fallback not present.
- Team cap not present.
- Catch-all handler present.
- Malformed-URL handling: confirmed in code.

7. Result after the fix

Metric (150-user burst)	Before	After
Poll p95 during burst	4.2 - 12.6 s	about 0.79 s
Registration p95	4.4 - 11.8 s	about 1.6 s
Registrations completed in 30 s	264 - 537	1639
Failures	0 - 17.8%	0%
Group Commit Run	register p95 4.54 s, poll p95 97.6 ms, 789 registrations, 0% failures; poll-only p95 1.92 ms. *Note: register threshold (p95 < 1 s) failed in that run.*	

I take this as supporting evidence that blocking code was the main cause, since the same server did several times more work with less waiting. It is not a clean comparison, because conditions were not controlled and the "after" run handled far more registrations.

Open question: Registration p95 rose from 1.6 s to 4.54 s after group commit. Not explained. Hypotheses: shared thread pool between hashing and file writes, `JSON.stringify` of a growing state, machine variance, closed-loop test effects. UV_THREADPOOL_SIZE experiment: not done.

Remaining slowness (hypothesis only): the poll response grows with the number of teams, because each poll rebuilds the standings and compresses the response synchronously. **[not measured]**

Our real event is much smaller than the test (13 teams, not 150 users hammering registration for 30 seconds). The realistic test is still to run. **[not measured]**

8. Decision: run locally, don't deploy on Vercel

Internet is not needed for the event. We only need a network so every device can open the site running on my laptop. If we deployed on Vercel, the app would break, because of how our storage works:

- Vercel runs code as serverless functions, possibly many copies at once.
- Our state lives in one process's memory plus one file. Multiple copies would each have their own state, so teams would see different games.
- There is no shared persistent disk, and timers (`setInterval`) don't fit short-lived functions.
- Vercel also only allows load testing on Enterprise plans (per Vercel's policy as I read it), so we couldn't even test it properly.

Options I considered:

Option	Why not (or why)
Vercel for the whole app	Breaks the in-memory state and file storage design
SQLite	Still a single file on one server, with better transactions but the same shared-storage problem
Supabase (hosted Postgres)	Would solve shared storage, but needs a major code migration right before the event
Frontend on Vercel, backend on my laptop	Needs an HTTPS tunnel, CORS changes, and adds internet as a point of failure
Node + `state.json` on a local network (chosen)	**Simple, works offline, and matches what we load tested**

I set a code freeze on Oct 22. Anything not tested by then is left out.

9. Decision: how devices connect

Final setup: TP-Link router from the HOD, phones connected to it. Planned: my laptop. Actual: Revanth's laptop. Rehearsal on Oct 23-24: happened, found that deliberate server restarts recovered cleanly.

10. What happened at the event

- Rounds 1 and 2 were configured at 30 minutes on the laptop that ran the server; teams used about 20, and I closed both rounds from the admin dashboard.
- The server ran on a teammate's local copy, not on main, because my laptop failed to start it under stage pressure.
- Round 3 used the buzzer console: 8 finalists, 10 questions, first buzzer +50, rebound team +35 with the first team's wrong option removed.
- **Lessons:** deploy from a tag, one start command, rehearse on the event laptop, show commit hash and round lengths on the admin screen.

- **Buzzer fairness:** three teams won most buzzes (my recollection, not from logs). Cause not checked yet.

11. Things I learned

- **Measure before fixing.** My first guess was plausible, but the numbers (3 ms vs seconds) are what pointed at a blocked event loop.
- **One test run is not a baseline.** Variance between runs was big.
- **Making code async adds new bugs** (races between await points and between overlapping writes), so each fix needs its own safety rule.
- **The simplest setup** that fits the real constraints beat the more "modern" deployment.
- **Friends' questions improved the design** (overlapping writes, disk failures, backpressure).

12. Ideas for later (Proposed, not built)

- Move state to PostgreSQL if we ever need several server instances.
- Cache the standings for a short time and compress asynchronously, if polling slows down with many teams.
- Redis for caching and rate limiting.
- WebSockets instead of polling for live updates.
- Docker for repeatable deployment.
- An answer-submission load test.
- Try Bun as a controlled experiment after the event.
- Kafka and similar tools solve scale problems we don't have.
- Buzzer fairness check (open timestamp and poll-gating).
- fsync, metrics endpoint, tests and CI.